

Capítulo 09 — Arquitecturas Multiagente

Sección 01 — Introducción a las arquitecturas multiagente

Un agente único, bien diseñado, puede resolver una clase sorprendentemente amplia de problemas. Puede razonar, consultar herramientas, mantener memoria episódica y ejecutar planes de múltiples pasos. Los capítulos anteriores de este módulo construyeron ese agente capa por capa: primero el contexto, luego la memoria, después las herramientas, finalmente los patrones de coordinación. Ese agente es poderoso.

Pero hay una categoría de problemas que un agente único no puede resolver bien, no porque le falte capacidad de razonamiento, sino porque la naturaleza del problema excede los límites estructurales de un sistema secuencial con una sola ventana de contexto y un único hilo de ejecución.

¿Cuáles son esos problemas? Son exactamente cuatro tipos:

Problemas que requieren paralelismo real. Cuando una tarea puede descomponerse en subtarear independientes que deben completarse simultáneamente para que el resultado sea oportuno, un agente único actúa en serie donde el sistema necesita actuar en paralelo. Analizar veinte documentos de forma concurrente, ejecutar tres búsquedas independientes al mismo tiempo, generar variantes alternativas de una propuesta simultáneamente: estos son casos donde la arquitectura secuencial impone una latencia innecesaria.

Problemas que requieren especialización profunda. Un agente generalista que intenta hacer bien muchas cosas distintas necesita un prompt de sistema extenso, herramientas diversas y un contexto que mezcla instrucciones de naturaleza muy diferente. La calidad de sus outputs en cada dominio es inevitablemente menor que la de un agente cuyo único propósito es ese dominio específico, con un prompt ajustado a esa tarea y herramientas seleccionadas para ese trabajo. Un agente especializado en análisis jurídico con acceso a bases de datos legales produce outputs más precisos que un agente generalista que también hace análisis jurídico entre otras cosas.

Problemas que requieren verificación independiente. Cuando la corrección de un output es crítica —un diagnóstico médico, un contrato legal, una recomendación financiera—, un único agente que genera y valida su propio trabajo tiene un conflicto de interés estructural: tiende a confirmar lo que produjo. Un segundo agente que recibe el output del primero como input a evaluar, sin haber participado en su generación, aplica una perspectiva genuinamente independiente. Este es el patrón de Reflexión a escala de sistema.

Problemas cuya escala excede la ventana de contexto de un único agente. Cuando el volumen de información que una tarea requiere procesar supera lo que puede mantenerse en una sola ventana de contexto, la única solución que no implica compresión con pérdida es distribuir la tarea entre múltiples agentes, cada uno operando sobre un fragmento manejable del problema total.

Cuando uno o más de estos cuatro factores están presentes, la arquitectura de múltiples agentes no es una complejidad opcional: es la respuesta correcta al problema.

Qué es un sistema multiagente

Un sistema multiagente es una arquitectura en la que dos o más agentes de IA coordinan su trabajo para completar una tarea que ninguno de ellos podría completar de forma óptima en solitario. Cada agente tiene su propio contexto, su propia instrucción de sistema, su propio conjunto de herramientas y, potencialmente, su propia memoria. Lo que los conecta es un mecanismo de comunicación —un protocolo, un almacén de estado compartido, o ambos— y una lógica de coordinación que determina quién hace qué y en qué orden.

Esto no es lo mismo que llamar a un modelo de lenguaje varias veces en un pipeline. En un pipeline secuencial, el output de una llamada es el input de la siguiente, pero no hay agentes en el sentido estricto: hay funciones que se encadenan. Un sistema multiagente implica agentes con autonomía propia —capaces de tomar decisiones, usar herramientas, manejar errores y reportar resultados— coordinados por una lógica que puede ser tan simple como un orquestador central o tan compleja como un protocolo de negociación entre pares.

La progresión natural desde el agente único

El Capítulo 08 construyó el agente individual: la unidad mínima de comportamiento autónomo en un sistema de IA. Este capítulo escala esa unidad hacia sistemas compuestos.

La progresión no es arbitraria. Es la misma que ocurre en ingeniería de software cuando un monolito bien diseñado alcanza los límites de lo que una única unidad de despliegue puede manejar: no se abandona el monolito porque sea mal código, sino porque la escala del

problema cambió. El agente multiagente es al agente único lo que los microservicios son al monolito bien estructurado: una respuesta arquitectónica a la complejidad del problema, no una señal de que el diseño anterior era incorrecto.

Lo que este capítulo cubre

Las quince secciones de este capítulo construyen el marco completo para diseñar, coordinar y operar sistemas multiagente:

- La sección 02 establece los criterios de decisión para saber cuándo una arquitectura multiagente es la respuesta correcta y cuándo es sobreingeniería.
- Las secciones 03 y 04 desarrollan cómo se diseñan los agentes individuales dentro del sistema y cómo colaboran entre sí.
- Las secciones 05 y 06 abordan los mecanismos de comunicación y la arquitectura de orquestación.
- Las secciones 07 y 08 tratan los problemas de ingeniería más desafiantes: la consistencia del estado compartido y la tolerancia a fallos.
- La sección 09 sistematiza los patrones que funcionan y los anti-patrones que deben evitarse.
- La sección 10 aplica todo lo anterior a un caso empresarial completo.
- Las secciones 11 a 15 proveen el laboratorio práctico, la checklist operativa, el resumen, la autoevaluación y la transición al capítulo siguiente.

Al completar este capítulo, el lector podrá identificar con precisión qué tipo de problema justifica una arquitectura multiagente, diseñar la topología apropiada para ese problema, definir los roles y mecanismos de comunicación de los agentes, anticipar los puntos de fallo del sistema y aplicar estrategias concretas para hacerlo resiliente.

La sección 02 comienza por la decisión más importante del capítulo: ¿cuándo realmente necesitamos más de un agente? La respuesta correcta a esa pregunta evita la complejidad innecesaria que convierte un sistema potencialmente elegante en un sistema difícil de mantener.

Capítulo 09 — Arquitecturas Multiagente

Sección 02 — ¿Cuándo utilizar múltiples agentes?

La arquitectura multiagente es una herramienta poderosa. También es una fuente frecuente de sobreingeniería. La distinción entre ambos usos no está en la tecnología sino en el problema: algunos problemas exigen múltiples agentes, la mayoría no. Un AI Engineer que no puede hacer esa distinción construye sistemas innecesariamente complejos cuando el problema pedía un agente simple, o sistemas inevitablemente frágiles cuando el problema pedía un sistema distribuido.

Esta sección establece los criterios de decisión. No como reglas rígidas, sino como un árbol de preguntas que, respondidas honestamente, señalan la arquitectura correcta.

El punto de partida: empieza siempre con el agente único

Antes de cualquier análisis, la regla de base es esta: **diseña primero con un agente único**. No porque sea siempre suficiente, sino porque es siempre más simple, más barato, más fácil de depurar y más rápido de iterar. Si el agente único puede resolver el problema con calidad aceptable, ese es el sistema correcto. La complejidad adicional de una arquitectura multiagente tiene un costo real: en latencia, en costo de tokens, en superficie de errores, en dificultad operativa.

El multiagente se justifica cuando el agente único encuentra límites que no puede superar estructuralmente. No cuando "sería más elegante", no cuando "escalaría mejor en teoría", sino cuando el problema real impone restricciones que el agente único no puede satisfacer.

Las cuatro preguntas de decisión

Primera pregunta: ¿puede la tarea descomponerse en subtareas independientes que se beneficiarían de ejecución simultánea?

Si la respuesta es sí, el paralelismo de agentes reduce la latencia total del sistema. Considera un sistema de due diligence empresarial que debe analizar estados financieros, reputación legal, historial de clientes y cobertura mediática de una compañía. Cada análisis es

independiente de los demás. Un agente único los haría en secuencia: primero financiero, luego legal, luego clientes, luego medios. Cuatro agentes especializados los hacen en paralelo. La latencia total pasa de ser la suma de las cuatro duraciones a ser la duración del análisis más lento. Cuando la tarea tiene esta estructura —subtareas independientes cuyo resultado no depende entre sí—, el paralelismo de agentes es directamente útil.

Si las subtareas son secuenciales (el output de A es el input de B), el paralelismo no ayuda. Un pipeline secuencial entre dos llamadas al modelo suele ser suficiente.

Segunda pregunta: ¿requieren distintas partes de la tarea conocimiento especializado, herramientas o instrucciones de sistema fundamentalmente distintas?

Un agente generalista que analiza código en Python, redacta contratos legales y responde consultas médicas necesita un prompt de sistema que intente cubrir todos esos dominios simultáneamente. El resultado es predecible: el agente es mediocre en todos ellos porque ninguno recibe el tratamiento especializado que merece.

La especialización de agentes es la respuesta correcta cuando los dominios de trabajo son suficientemente distintos como para que un único prompt de sistema no pueda servir a todos con la misma precisión. Un agente cuya única función es analizar código Python, con herramientas de ejecución de código y acceso a documentación técnica, produce análisis fundamentalmente mejores que el agente generalista. Si el problema tiene esa estructura —múltiples dominios que requieren especialización real—, la arquitectura multiagente con agentes especializados es la elección correcta.

Tercera pregunta: ¿la corrección del output es suficientemente crítica como para requerir verificación independiente?

Un agente que genera una conclusión y la valida por sí mismo tiene un sesgo estructural hacia confirmar lo que produjo. Para tareas de baja criticidad, este sesgo es aceptable. Para tareas donde un error tiene consecuencias significativas —una recomendación de inversión, un diagnóstico diferencial, la verificación de código de seguridad—, la verificación independiente por un segundo agente que no participó en la generación añade una capa de robustez real.

El patrón de Reflexión en arquitectura multiagente es precisamente esto: un agente generador y un agente crítico que evalúa el output del primero contra criterios explícitos antes de que ese output sea aceptado. La criticidad de la tarea es el factor decisivo: si el costo de un error justifica la latencia y el costo adicional de la verificación, la arquitectura dual es correcta.

Cuarta pregunta: ¿el volumen de información que la tarea requiere procesar excede lo que puede mantenerse en una sola ventana de contexto sin compresión con pérdida inaceptable?

Cuando la respuesta es sí, la distribución entre agentes es la única solución que preserva la fidelidad de la información. Un análisis que requiere procesar doscientos documentos densos no puede hacerse con calidad en una única ventana de contexto de cien mil tokens. Dos agentes procesando cien documentos cada uno, con un agente síntesis que combina sus outputs, distribuyen la carga sin sacrificar fidelidad. Este criterio es objetivo y medible: si el contenido de la tarea cabe en la ventana de contexto con margen para el razonamiento, el agente único es suficiente; si no cabe, el multiagente es necesario.

El árbol de decisión

```
¿La tarea tiene subtareas independientes paralelizables?  
→ SÍ: considerar agentes paralelos  
→ NO ↓  
  
¿Los dominios de la tarea requieren especialización radicalmente distinta?  
→ SÍ: considerar agentes especializados  
→ NO ↓  
  
¿La criticidad del output justifica verificación independiente?  
→ SÍ: considerar patrón generador + crítico  
→ NO ↓  
  
¿El volumen de información excede la ventana de contexto sin pérdida aceptable?  
→ SÍ: considerar agentes distribuidores + síntesis  
→ NO → Agente único es la arquitectura correcta
```

Si ninguna de las cuatro preguntas produce un "sí", la arquitectura de agente único es la respuesta correcta. No como segunda opción: como la mejor opción para ese problema.

Anti-patrón: multiagente por aspiración arquitectónica

El anti-patrón más frecuente en este campo no es técnico sino motivacional: equipos que adoptan arquitecturas multiagente porque son más sofisticadas, porque los demos se ven más

impresionantes, o porque "ya que estamos construyendo con IA, hagámoslo bien". Ninguna de estas razones es una razón válida para añadir la complejidad de un sistema multiagente.

El síntoma más claro de este anti-patrón es la dificultad para responder concretamente cuál de las cuatro preguntas anteriores justificó la arquitectura. Si la respuesta es "porque es más escalable" o "porque es la arquitectura moderna", el sistema probablemente debería ser más simple.

El costo real del multiagente

Antes de decidir implementar una arquitectura multiagente, el AI Engineer debe estimar con honestidad:

- **Latencia:** cada agente adicional añade tiempo de procesamiento. Los sistemas paralelos reducen la latencia del caso crítico, pero añaden overhead de coordinación. Los sistemas secuenciales con múltiples agentes acumulan latencia en cada paso.
- **Costo de tokens:** cada agente consume tokens de input y output. Un sistema de cuatro agentes que procesan el mismo contexto base multiplica ese costo base por cuatro.
- **Superficie de errores:** cada agente es un punto de fallo potencial. Cada canal de comunicación entre agentes es otro. La complejidad del sistema de manejo de errores crece con el número de agentes.
- **Dificultad de depuración:** cuando un sistema multiagente produce un output incorrecto, identificar cuál agente introdujo el error y en qué momento requiere trazabilidad completa. Sin ella, el sistema es una caja negra difícil de mantener.

Estos costos no son razones para evitar el multiagente cuando el problema lo requiere. Son razones para no adoptarlo cuando el problema no lo requiere.

La sección 03 asume que la decisión ya fue tomada: el problema justifica una arquitectura multiagente. El siguiente paso es diseñar los agentes individuales que componen ese sistema, comenzando por la pregunta de cómo definir roles y especialización de forma que el sistema sea más que la suma de sus partes.

Capítulo 09 — Arquitecturas Multiagente

Sección 03 — Roles y especialización de agentes

Un sistema multiagente que distribuye trabajo entre agentes genéricos no es un sistema multiagente bien diseñado: es un sistema que añade la complejidad de la coordinación sin aprovechar la principal ventaja de la arquitectura, que es precisamente la especialización. La potencia del diseño multiagente está en que cada agente hace una cosa con precisión, no en que muchos agentes hagan muchas cosas con mediocridad distribuida.

Esta sección desarrolla cómo se definen los roles de los agentes dentro de un sistema, qué significa especializar un agente de forma efectiva y qué decisiones de diseño determinan la calidad de cada unidad dentro del sistema compuesto.

Qué define el rol de un agente

El rol de un agente no es un nombre en un diagrama. Es la combinación de tres elementos que deben estar alineados de forma coherente:

El alcance funcional: qué hace el agente, en términos concretos y limitados. "Analiza el código Python enviado y detecta vulnerabilidades de seguridad según las categorías del OWASP Top 10" es un alcance funcional bien definido. "Ayuda con el desarrollo de software" no lo es.

El conjunto de herramientas: qué herramientas tiene disponibles. Un agente cuyo rol es analizar código necesita acceso a herramientas de ejecución de código, linters, bases de datos de vulnerabilidades conocidas. No necesita herramientas de búsqueda web general, acceso a calendarios ni capacidades de redacción de documentos. La disciplina de incluir solo las herramientas que el rol específicamente requiere evita que el agente las use de forma inapropiada y reduce la superficie de errores.

La instrucción de sistema: el prompt de sistema que define la identidad, los objetivos, las restricciones y el formato de output del agente. Esta instrucción debe ser específica para el rol, no una versión recortada de un prompt generalista. Un agente crítico tiene una instrucción de sistema fundamentalmente distinta a la de un agente generador, incluso si trabajan sobre el

mismo dominio. El agente generador está orientado a producir; el agente crítico está orientado a evaluar. Sus instrucciones de sistema deben reflejar esa diferencia.

Roles típicos en sistemas multiagente de producción

Los sistemas multiagente bien diseñados tienden a converger en un conjunto de roles arquetípicos que aparecen en distintas configuraciones según el dominio:

El agente orquestador. No produce contenido directamente. Descompone tareas complejas en subtarear, asigna esas subtarear a los agentes apropiados, recibe sus outputs y sintetiza el resultado final. Su instrucción de sistema lo orienta explícitamente hacia la coordinación, no hacia la ejecución. Es el director del sistema, no uno de sus instrumentistas.

El agente investigador. Su función es recuperar información: buscar en bases de conocimiento, consultar APIs externas, recuperar documentos relevantes. Está equipado con herramientas de búsqueda y recuperación. Su output es información estructurada para ser consumida por otros agentes, no una respuesta final para el usuario.

El agente analista. Recibe información —producida por el investigador u obtenida de otra fuente— y produce análisis: patrones, inferencias, evaluaciones, predicciones. Sus herramientas son de cómputo y razonamiento, no de búsqueda. No necesita saber cómo se recuperó la información que analiza.

El agente redactor. Transforma análisis o información estructurada en texto de alta calidad para el output final: informes, respuestas al usuario, documentos. Su instrucción de sistema está orientada a la claridad, el tono y el formato. No necesita acceso a bases de datos ni herramientas de cómputo.

El agente crítico. Recibe el output de otro agente —generalmente el redactor o el analista— y lo evalúa contra criterios explícitos. Detecta errores de razonamiento, inconsistencias, afirmaciones no respaldadas, problemas de tono o formato. Su output es una evaluación estructurada, no una versión mejorada del texto: la corrección es responsabilidad del agente que generó el output, no del crítico.

El agente ejecutor. Es el único agente del sistema que tiene permiso para realizar acciones con efectos en el mundo externo: enviar correos, actualizar registros, ejecutar código en producción. Esta concentración de capacidad de acción en un único agente especializado es una decisión de seguridad deliberada: facilita el control de autorización y la auditoría de todas las acciones del sistema.

El principio de mínima capacidad

Un agente bien diseñado tiene las capacidades mínimas necesarias para cumplir su función. Este principio no es una restricción arbitraria: es la condición que permite que el sistema sea predecible, seguro y depurable.

Un agente investigador que también puede ejecutar código en producción es un punto de riesgo: si el agente es comprometido o produce un error de razonamiento, las consecuencias pueden ser irreversibles. Un agente investigador que solo puede buscar y leer información tiene un radio de daño acotado.

El principio de mínima capacidad también mejora la calidad del razonamiento. Un agente que tiene disponibles veinte herramientas diferentes dedica esfuerzo de razonamiento a decidir cuál usar. Un agente que tiene tres herramientas relevantes para su función específica utiliza ese mismo esfuerzo de razonamiento en hacer bien su trabajo.

Diseñar el rol antes de implementar el agente

El error más común en el diseño de agentes especializados es comenzar por la implementación: "creo un agente, le doy herramientas, le pongo un prompt". El resultado es un agente cuyo rol emerge de forma ad hoc de las herramientas que se le dieron y del prompt que se le ocurrió al momento de escribirlo.

El proceso correcto es inverso: primero se define el rol con precisión (¿qué hace este agente? ¿qué no hace?), luego se seleccionan las herramientas que ese rol específicamente requiere, y finalmente se escribe la instrucción de sistema que expresa ese rol con precisión. El orden importa porque la claridad del rol determina la coherencia de las decisiones de implementación que le siguen.

Una prueba útil al diseñar el rol de un agente es la pregunta de la sustitución: si este agente fuera un ser humano contratado para ese rol específico, ¿qué haría en su primer día de trabajo? ¿Qué recursos necesitaría? ¿Qué haría y qué explícitamente no haría? Si esa descripción es clara, el rol está bien definido. Si la respuesta es vaga o incluye demasiadas responsabilidades heterogéneas, el rol necesita más delimitación.

La frontera entre agentes y su impacto en la calidad

La calidad de un sistema multiagente depende tanto de lo que hace cada agente como de dónde termina la responsabilidad de uno y comienza la del siguiente. Las fronteras mal

definidas producen solapamientos (dos agentes intentan hacer lo mismo con enfoques distintos) o vacíos (ningún agente es responsable de una tarea necesaria).

Las fronteras se definen por los contratos de input y output: qué formato produce el agente A como output y qué formato espera consumir el agente B como input. Cuando ambos formatos coinciden sin necesidad de transformación, la frontera está bien definida. Cuando el agente B necesita interpretar o transformar el output del agente A antes de poder trabajar con él, la frontera necesita revisión: o el formato de output del agente A debe ser más preciso, o la responsabilidad de transformación debe asignarse explícitamente a alguno de los dos.

La sección 04 toma los agentes bien diseñados de esta sección y examina cómo se organizan entre sí. Las topologías de colaboración —jerárquica, en pipeline, entre pares, basada en mercado— no son variantes estéticas: cada una tiene casos de uso óptimos y compromisos específicos que el arquitecto debe evaluar antes de comprometerse con una.

Capítulo 09 — Arquitecturas Multiagente

Sección 04 — Patrones de colaboración entre agentes

La especialización de cada agente responde a la pregunta de qué hace cada unidad del sistema. La topología responde a la pregunta de cómo se relacionan entre sí. Estas dos preguntas son independientes: se pueden tener agentes perfectamente especializados organizados en una topología inadecuada para el problema, y el sistema fallará por la desconexión entre diseño de unidades y diseño de sistema.

Hay cuatro topologías fundamentales en los sistemas multiagente. No son mutuamente excluyentes —los sistemas reales con frecuencia combinan varias— pero cada una tiene una lógica propia y un conjunto de casos de uso donde es la elección natural.

Topología jerárquica

En la topología jerárquica existe un agente en el nivel superior —el orquestador— que tiene autoridad sobre todos los demás. El orquestador recibe la tarea del sistema, la descompone en subtarear, asigna cada subtarea al agente especializado correspondiente, recibe los resultados y produce la síntesis final. Los agentes del nivel inferior no se comunican entre sí directamente: toda comunicación pasa por el orquestador.

Esta topología es la arquitectura de referencia para sistemas multiagente empresariales porque tiene propiedades que facilitan su operación: la responsabilidad está centralizada (el orquestador es el único punto de coordinación), el flujo de datos es predecible (todo pasa por el orquestador), y el debug es relativamente directo (se puede examinar qué recibió y qué envió el orquestador en cada paso).

Sus compromisos son igualmente claros: el orquestador es un cuello de botella potencial. Si el orquestador falla, el sistema completo falla. Si el orquestador toma decisiones de descomposición incorrectas, los agentes especializados las ejecutan perfectamente pero producen el resultado incorrecto. La calidad del orquestador determina la calidad del sistema.

Casos de uso óptimos: sistemas con tareas bien estructurables en subtarear, donde la coordinación centralizada es una ventaja de control más que una limitación de rendimiento.

Procesamiento de documentos, generación de informes complejos, pipelines de análisis multidimensional.

Topología en pipeline

En el pipeline, los agentes se organizan en secuencia lineal. El agente A procesa el input inicial y produce un output que el agente B consume como input, produciendo a su vez un output que el agente C consume, y así sucesivamente. No hay un orquestador central: la coordinación está implícita en la secuencia y en los contratos de input/output entre agentes consecutivos.

El pipeline es la topología más simple de implementar y razonar sobre ella. Cuando la tarea tiene una estructura naturalmente secuencial —donde la fase B genuinamente no puede comenzar hasta que la fase A termine— el pipeline no es solo simple sino correcto. Intentar paralelizar un proceso inherentemente secuencial solo añade complejidad sin reducir la latencia total.

Sus limitaciones son simétricas a su simpleza: no hay paralelismo posible dentro de la topología pura. Si el agente C falla, el pipeline completo se detiene. Y si el error fue introducido en el agente A, el agente B lo amplifica y el agente C lo amplifica aún más: los errores se propagan hacia adelante sin oportunidad de corrección.

Casos de uso óptimos: transformaciones secuenciales donde cada etapa agrega valor sobre el output de la anterior sin posibilidad de paralelismo. Extracción → normalización → análisis → reporte. Transcripción → traducción → resumen → indexación.

Topología entre pares (peer-to-peer)

En la topología entre pares, los agentes pueden comunicarse directamente entre sí sin un coordinador central. Cada agente conoce la existencia de otros agentes y puede iniciar comunicación con cualquiera de ellos cuando lo necesita. La coordinación emerge de la interacción descentralizada más que de una autoridad central.

Esta topología tiene capacidades que las anteriores no pueden replicar: permite que un agente solicite ayuda a otro sin que el orquestador deba anticipar esa necesidad. Un agente analista que descubre que necesita información adicional puede solicitarla directamente al agente investigador, sin esperar que el orquestador detecte esa necesidad y la gestione. Esto produce sistemas más adaptativos y capaces de manejar casos que no fueron completamente anticipados en el diseño.

El costo es la complejidad: los sistemas entre pares son notablemente más difíciles de depurar, de razonar sobre su comportamiento y de garantizar su terminación. Pueden surgir ciclos de comunicación, deadlocks donde dos agentes esperan mutuamente una respuesta y estados del sistema difíciles de predecir. La trazabilidad de quién le dijo qué a quién y en qué orden requiere instrumentación cuidadosa.

Casos de uso óptimos: sistemas de investigación abierta donde los agentes necesitan consultar a otros en función de lo que descubren, sistemas de deliberación colaborativa y cualquier caso donde la rigidez de un orquestador central impide la adaptación necesaria.

Topología basada en mercado

En la topología de mercado, los agentes compiten o cooperan por recursos: tareas, datos o capacidad de procesamiento. Un agente "publicador" anuncia que tiene una tarea disponible. Los agentes "suscriptores" que tienen la capacidad apropiada postulan para ejecutarla. El sistema asigna la tarea al agente más disponible o más calificado según algún criterio de selección.

Esta topología es la más compleja pero también la más escalable. Es el modelo correcto cuando el volumen de trabajo es variable e impredecible, cuando hay múltiples instancias de agentes del mismo tipo que pueden ejecutar la misma clase de tarea, y cuando se quiere distribuir la carga dinámicamente en función de la disponibilidad real del sistema.

Es también la más difícil de implementar correctamente: requiere un mecanismo de descubrimiento de agentes, un protocolo de postulación, un mecanismo de asignación y un sistema de confirmación y reporte de resultados. Para la mayoría de sistemas empresariales de tamaño mediano, esta complejidad no está justificada.

Casos de uso óptimos: sistemas de alta escala con trabajo altamente variable, múltiples instancias de agentes especializados y necesidad de distribución dinámica de carga.

Patrones híbridos: la práctica real

Los sistemas reales rara vez implementan una topología pura. La combinación más común es una topología jerárquica que incorpora elementos de pipeline en el nivel de los agentes ejecutores: el orquestador coordina en el nivel superior, y los agentes especializados ejecutan flujos secuenciales internamente.

Otra combinación frecuente es la topología jerárquica con un sub-grafo de pares: el orquestador coordina los macro-pasos del sistema, pero dentro de un micro-sistema de

análisis colaborativo, un conjunto de agentes especializados se consulta mutuamente para producir un análisis más completo.

El criterio para combinar topologías es el mismo que para elegir entre ellas: cada nivel del sistema debe tener la topología que mejor responde a su naturaleza. La coordinación de alto nivel tiende a beneficiarse de la jerarquía; la ejecución de tareas específicas puede beneficiarse del pipeline; los sistemas de exploración o deliberación se benefician de la topología entre pares.

Documentar la topología antes de implementarla

Una práctica indispensable antes de comenzar la implementación de cualquier sistema multiagente es dibujar el diagrama de la topología con el nivel de detalle que incluya: cada agente con su rol, las direcciones de comunicación entre ellos, y los formatos de los mensajes que transitan cada conexión. Este diagrama no es documentación posterior a la implementación: es el artefacto de diseño previo a la primera línea de código.

El diagrama de topología cumple dos funciones. La primera es la verificación de coherencia: si el diagrama muestra que el agente A necesita información que produce el agente C pero C es posterior a A en el pipeline, hay un error de diseño que el diagrama hace visible antes de que sea un error de código. La segunda función es la comunicación: un equipo que trabaja sobre el mismo sistema necesita un modelo compartido de cómo funciona, y ese modelo es el diagrama.

Con los agentes diseñados y la topología elegida, la siguiente pregunta es mecánica pero crucial: ¿cómo se comunican concretamente los agentes entre sí? La sección 05 desarrolla los protocolos y mecanismos de comunicación que hacen que la topología funcione en la práctica.

Capítulo 09 — Arquitecturas Multiagente

Sección 05 — Coordinación, comunicación y protocolos

La topología define la estructura del sistema. La comunicación es lo que hace que esa estructura funcione. Dos sistemas con topologías idénticas pueden tener comportamientos radicalmente distintos si sus mecanismos de comunicación difieren. Y los errores de comunicación entre agentes —mensajes malformados, mensajes perdidos, mensajes entregados en el orden incorrecto— son una de las causas más frecuentes de fallo en sistemas multiagente en producción.

Esta sección desarrolla cómo se comunican los agentes: qué formatos de mensaje funcionan, qué modelos de comunicación existen y cuándo usar cada uno, y cómo diseñar la comunicación de forma que los errores sean detectables y recuperables.

El mensaje como unidad fundamental

En un sistema multiagente, toda interacción entre agentes ocurre a través de mensajes. Un mensaje es una unidad de información que un agente envía a otro, con la intención de transferir datos, solicitar una acción o reportar un resultado. Que el mensaje viaje directamente de un agente a otro, a través de un bus de mensajes, o mediante un estado compartido es un detalle de implementación. Lo que no es un detalle es el contenido y la estructura del mensaje.

Un mensaje bien diseñado tiene cuatro componentes:

El tipo: qué clase de mensaje es. Un mensaje puede ser una tarea (solicitud de que el agente receptor ejecute algo), un resultado (output de una tarea completada), un error (reporte de que algo falló), o un estado (actualización sobre el progreso de una tarea en curso). El tipo determina cómo el agente receptor debe procesar el mensaje.

El identificador: un identificador único que permite correlacionar un resultado o un error con la tarea que lo originó. Sin identificador, un sistema que ejecuta múltiples tareas en paralelo no puede saber a qué solicitud corresponde cada respuesta.

El payload: el contenido propio del mensaje. Para una tarea, el payload es la descripción de la tarea a ejecutar con todos los datos que el agente receptor necesita. Para un resultado, el payload es el output estructurado de la tarea. Para un error, el payload es la descripción del error con suficiente contexto para que el agente coordinador pueda decidir cómo responder.

Los metadatos: información de contexto que el agente receptor puede necesitar sin que sea parte del payload principal. El agente de origen, la marca temporal, la prioridad, el número de reintentos realizados.

Formatos de mensaje: texto natural vs. estructura formal

Una decisión de diseño frecuente en sistemas multiagente es si los mensajes entre agentes deben estar en lenguaje natural o en un formato estructurado como JSON.

El lenguaje natural entre agentes parece intuitivo: los modelos de lenguaje son buenos procesando texto. El problema es que introduce ambigüedad donde se necesita precisión. Cuando el agente A le dice al agente B "analiza el documento y dime qué contraste", el agente B necesita inferir qué tipo de análisis, con qué criterios, en qué formato debe devolver el resultado. Si ese análisis es parte de un sistema automatizado donde el output del agente B será procesado por el agente C programáticamente, la variabilidad en el formato del output puede romper el procesamiento.

Los mensajes estructurados en JSON eliminan esa ambigüedad. Un mensaje que especifica

```
{"tarea": "analisis_sentimiento", "texto": "...", "categorias": ["positivo", "negativo", "neutro"], "formato_salida": "json_con_puntuacion"}
```

no deja lugar a interpretación. El agente receptor sabe exactamente qué debe hacer, con qué datos y en qué formato debe responder.

La regla práctica: usa lenguaje natural cuando el receptor es humano. Usa formatos estructurados cuando el receptor es otro agente o código que procesará el resultado. Esta regla no impide que un agente procese texto en lenguaje natural como parte del payload — puede hacerlo— pero los metadatos, el tipo, el identificador y los parámetros de la tarea deben estar en formato estructurado.

Comunicación síncrona vs. asíncrona

El modelo síncrono de comunicación funciona así: el agente A envía un mensaje al agente B y espera. No hace nada más hasta recibir la respuesta. El modelo asíncrono funciona así: el agente A envía un mensaje al agente B, continúa su propio procesamiento, y maneja la respuesta del agente B cuando esta llega, sin haber bloqueado su ejecución en el intervalo.

La elección entre ambos modelos depende de si el agente A necesita el resultado del agente B para continuar su propio trabajo:

Comunicación síncrona: cuando el agente A no puede avanzar sin el resultado del agente B. El orquestador que espera el resultado de todos los agentes especializados antes de sintetizar la respuesta final opera de forma síncrona con cada agente en el sentido de que necesita todos los resultados para producir la síntesis.

Comunicación asíncrona: cuando el agente A puede continuar con otras tareas mientras el agente B trabaja. Un agente investigador que lanzó cinco búsquedas en paralelo no tiene razón para bloquear su ejecución esperando la primera: puede continuar otras actividades y procesar las respuestas a medida que llegan.

Los sistemas reales con frecuencia combinan ambos modelos: el orquestador lanza todas las tareas de forma asíncrona (no espera a que la primera termine para lanzar la segunda), pero luego espera de forma síncrona hasta tener todos los resultados antes de sintetizar.

El bus de mensajes y el estado compartido

Hay dos mecanismos fundamentales para implementar la comunicación entre agentes:

El bus de mensajes (message bus): los agentes publican mensajes en un canal compartido y se suscriben a los canales que les interesan. Un agente que completa una tarea publica el resultado en el canal de resultados. El agente que inició la tarea escucha ese canal y procesa el resultado cuando llega. El bus de mensajes desacopla a los agentes: el emisor no necesita saber quién va a procesar su mensaje, y el receptor no necesita saber quién lo originó.

El estado compartido (shared state store): existe un almacén de estado centralizado al que todos los agentes tienen acceso de lectura y escritura. Los agentes leen el estado para obtener contexto, ejecutan su trabajo y escriben sus resultados en el estado. Otros agentes leen esos resultados y actúan sobre ellos. El estado compartido hace que toda la información del sistema esté en un lugar, lo que facilita la visibilidad y la depuración, pero introduce problemas de concurrencia que la sección 07 trata con detalle.

En la práctica, muchos sistemas combinan ambos: el bus de mensajes para la coordinación de tareas (quién debe hacer qué) y el estado compartido para los datos de larga duración (el contexto acumulado que todos los agentes necesitan consultar).

Diseño para la resiliencia comunicacional

Un mensaje que llega mal formado, que nunca llega, o que llega dos veces son escenarios que el sistema debe manejar sin colapsar. El diseño de comunicación resiliente implica:

Validación en la entrada: cada agente que recibe un mensaje debe validar su estructura y contenido antes de procesarlo. Un mensaje que no cumple el esquema esperado debe rechazarse inmediatamente con un mensaje de error claro, no intentar procesarse de forma tolerante.

Idempotencia: si un mensaje se entrega dos veces —por un error del sistema de mensajería— el procesamiento debe producir el mismo resultado en ambas entregas. La idempotencia se logra verificando el identificador del mensaje antes de procesarlo: si ya fue procesado, el resultado se devuelve sin reejecutar el trabajo.

Confirmación de recepción: el emisor de un mensaje debe poder saber si el receptor lo recibió y lo procesó. Sin confirmación, un fallo silencioso puede dejar al emisor esperando indefinidamente un resultado que nunca llegará.

Timeouts: todo mecanismo de espera debe tener un tiempo máximo. Un agente que espera una respuesta que nunca llega debe tener un timeout después del cual reporta el error y permite que el sistema decida cómo continuar.

La sección 06 toma los mecanismos de comunicación de esta sección y los examina desde la perspectiva del control: cómo un agente planificador descompone las tareas del sistema, y cómo un agente supervisor garantiza la calidad de los resultados. Estos dos roles — planificador y supervisor— son los que elevan un conjunto de agentes a un sistema cohesivo.

Capítulo 09 — Arquitecturas Multiagente

Sección 06 — Planificadores y agentes supervisores

En un sistema multiagente que funciona correctamente, la inteligencia del sistema no reside únicamente en los agentes que ejecutan tareas. Reside también, y de forma crítica, en los agentes que deciden qué tareas ejecutar y que verifican que las tareas ejecutadas produjeron resultados correctos. Estos son los roles del planificador y del supervisor, y son los que transforman un conjunto de agentes independientes en un sistema con comportamiento coherente.

El planificador: de la tarea al plan

El planificador recibe una tarea compleja del nivel superior del sistema —ya sea del usuario directamente, de una aplicación o de otro agente de nivel superior— y la transforma en un plan: una secuencia ordenada de subtareas, cada una asignada a un agente específico, con las dependencias entre ellas explicitadas.

Esta función de descomposición es más difícil de lo que parece. Una buena descomposición tiene tres características:

Compleitud: el plan cubre todas las acciones necesarias para completar la tarea original. No hay subtareas implícitas que algún agente deba inferir o que queden sin asignación.

No redundancia: cada parte del trabajo está asignada a exactamente un agente. Cuando dos agentes hacen trabajo solapado, el resultado es ineficiencia y, peor, posibles inconsistencias si sus outputs sobre la misma pieza de trabajo no coinciden.

Corrección de dependencias: las dependencias entre subtareas están correctamente identificadas. Si la subtask B requiere el resultado de la subtask A, esa dependencia debe estar en el plan. Si A y B son independientes, el plan debe permitir su ejecución paralela.

El planificador implementa esta función usando el propio modelo de lenguaje como motor de razonamiento. A diferencia de un scheduler determinístico de software que ejecuta reglas predefinidas, el planificador de un sistema multiagente de IA puede razonar sobre tareas que

no fueron anticipadas en su diseño. Esta flexibilidad es una de las propiedades más valiosas del planificador basado en modelos de lenguaje: puede descomponer problemas que el diseñador del sistema no pudo prever completamente.

El riesgo simétrico es que el planificador puede generar planes incorrectos. Un modelo de lenguaje que razona sobre cómo descomponer una tarea puede hacerlo de forma plausible pero errónea: olvidar una dependencia, asignar una tarea al agente equivocado, o generar subtareas que no cubren completamente la tarea original. Este riesgo es la razón por la que el planificador y el supervisor no son el mismo agente.

Estructura de un plan

Un plan en un sistema multiagente tiene una representación formal, no textual. El planificador no produce un párrafo explicando qué deben hacer los agentes: produce una estructura de datos que el sistema puede interpretar y ejecutar. Una representación típica es una lista de nodos con sus dependencias:

```
{
  "plan_id": "plan_001",
  "tareas": [
    {
      "id": "t1",
      "agente": "investigador",
      "descripcion": "Recuperar los estados financieros del cliente X de los últimos tr",
      "dependencias": []
    },
    {
      "id": "t2",
      "agente": "investigador",
      "descripcion": "Recuperar el historial legal del cliente X en los últimos cinco a",
      "dependencias": []
    },
    {
      "id": "t3",
      "agente": "analista_financiero",
      "descripcion": "Analizar los estados financieros recuperados y producir un resume",
      "dependencias": ["t1"]
    },
    {
      "id": "t4",
      "agente": "analista_legal",
      "descripcion": "Analizar el historial legal y producir un resumen de riesgo",
      "dependencias": ["t2"]
    },
    {
      "id": "t5",
      "agente": "redactor",
      "descripcion": "Redactar el informe de due diligence integrando ambos análisis",
      "dependencias": ["t3", "t4"]
    }
  ]
}
```

```
]
}
```

Este plan codifica que t1 y t2 pueden ejecutarse en paralelo (sin dependencias), que t3 y t4 pueden ejecutarse en paralelo entre sí pero requieren que t1 y t2 hayan terminado, y que t5 solo puede comenzar cuando t3 y t4 estén completas. El ejecutor del plan —que puede ser el mismo planificador u otro componente del sistema— lee esta estructura y la ejecuta respetando las dependencias.

El supervisor: verificación independiente de calidad

El supervisor recibe el output de un agente ejecutor y lo evalúa contra criterios explícitos antes de que ese output sea aceptado como válido por el sistema. No produce contenido: produce evaluaciones.

La lógica de la supervisión es análoga a la revisión por pares en contextos humanos: la persona que generó el trabajo no es la mejor posición para detectar sus propios errores. El supervisor es el segundo par de ojos que puede detectar lo que el agente ejecutor no vio, precisamente porque no participó en la generación.

Un supervisor bien diseñado tiene criterios de evaluación explícitos en su instrucción de sistema. No evalúa de forma genérica ("¿esto es bueno?") sino contra dimensiones específicas relevantes para la tarea:

- Para un análisis financiero: ¿los números citados corresponden a las fuentes recuperadas? ¿Las conclusiones siguen lógicamente de los datos presentados? ¿Hay afirmaciones que no están respaldadas por evidencia?
- Para un bloque de código generado: ¿el código maneja los casos borde identificados en la especificación? ¿Hay dependencias no declaradas? ¿El código es ejecutable sin modificaciones?
- Para un informe redactado: ¿el tono es apropiado para la audiencia especificada? ¿La estructura sigue el formato requerido? ¿Hay inconsistencias entre diferentes secciones?

El output del supervisor es una evaluación estructurada que incluye: si el output es aceptable o requiere corrección, qué problemas específicos fueron encontrados y —opcionalmente— qué tipo de corrección resolvería cada problema. El supervisor no corrige: señala. La corrección es responsabilidad del agente que generó el output.

El ciclo planificador-ejecutor-supervisor

La arquitectura completa del sistema de control en un sistema multiagente de producción tiene la forma de un ciclo:

1. El planificador recibe la tarea y genera el plan de subtareas.
2. El ejecutor (orquestador) distribuye las subtareas a los agentes especializados según el plan.
3. Cada agente especializado ejecuta su subtaska y produce su output.
4. El supervisor recibe cada output y lo evalúa.
5. Si el output es aceptable, el ejecutor avanza al siguiente paso del plan.
6. Si el output requiere corrección, el ejecutor reenvía la subtaska al agente especializado con el feedback del supervisor.
7. El agente especializado produce una versión corregida.
8. El ciclo de supervisión se repite hasta que el output es aceptable o se alcanza un límite de reintentos.

Este ciclo añade latencia y costo en comparación con un sistema que acepta todos los outputs sin supervisión. El valor que justifica ese costo es la reducción de errores en el output final. Para tareas de baja criticidad donde el usuario puede evaluar y corregir el resultado, la supervisión puede omitirse. Para tareas de alta criticidad donde un error en el output tiene consecuencias reales, el ciclo planificador-ejecutor-supervisor es la arquitectura correcta.

Planificación adaptativa

En los sistemas multiagente más sofisticados, el planificador no genera un plan único al inicio y lo ejecuta de forma rígida. Genera un plan inicial, pero también tiene la capacidad de revisar ese plan cuando los resultados intermedios revelan información que cambia lo que se debe hacer.

Considera un sistema de investigación que planificó cinco búsquedas independientes. Las primeras dos búsquedas revelan que el problema tiene una dimensión no anticipada. Un planificador adaptativo puede actualizar el plan para añadir tres búsquedas adicionales que cubran esa dimensión nueva, antes de que el agente síntesis intente producir la respuesta final con información incompleta.

Esta capacidad de replanificación requiere que el planificador tenga acceso a los resultados intermedios del sistema y que su instrucción de sistema incluya explícitamente la responsabilidad de revisar y actualizar el plan cuando el contexto lo justifique. Sin esa

capacidad, el planificador es rígido y el sistema produce resultados de menor calidad en tareas que no se ajustan perfectamente al plan inicial.

La sección 07 aborda el problema técnico más desafiante de los sistemas multiagente: cómo múltiples agentes acceden y actualizan un estado compartido sin generar inconsistencias. Este no es un problema de coordinación de alto nivel sino un problema de ingeniería de sistemas que requiere soluciones concretas.

Capítulo 09 — Arquitecturas Multiagente

Sección 07 — Memoria compartida y contexto distribuido

Cuando un sistema multiagente trabaja en una tarea común, los agentes necesitan compartir información. El agente investigador necesita que el agente analista vea los documentos que recuperó. El agente analista necesita que el agente redactor vea las conclusiones que produjo. El agente supervisor necesita ver el output del agente que supervisa. Sin un mecanismo de compartición de información, los agentes trabajan en silos y el sistema no puede funcionar.

El mecanismo natural para resolver este problema es el estado compartido: un almacén de información al que todos los agentes pueden leer y escribir. Pero el estado compartido introduce uno de los problemas de ingeniería más clásicos de los sistemas distribuidos: ¿qué ocurre cuando dos agentes intentan modificar el mismo dato al mismo tiempo? ¿Cómo sabe un agente que la información que está leyendo sigue siendo válida y no fue modificada por otro agente después de que la leyó?

Estos no son problemas teóricos. En un sistema multiagente que opera en paralelo, son situaciones que ocurren regularmente. El diseño del mecanismo de memoria compartida determina si esas situaciones producen errores silenciosos, errores detectables o comportamiento correcto.

El problema de la concurrencia en el estado compartido

Considera un sistema multiagente que mantiene un objeto de estado con el contexto acumulado de la tarea. El estado incluye, entre otras cosas, una lista de conclusiones a las que los agentes han llegado durante el proceso.

El agente A lee la lista de conclusiones (tiene cuatro elementos), añade su conclusión y escribe la lista de vuelta al estado (ahora tiene cinco elementos). Simultáneamente, el agente B leyó la misma lista cuando tenía cuatro elementos (antes de que A la modificara), añadió su conclusión y escribió la lista de vuelta. El resultado es que la lista tiene cinco elementos: las cuatro originales más la de B, pero la conclusión de A se perdió porque B sobrescribió con la versión que leyó antes de la modificación de A.

Esto es una condición de carrera clásica. En sistemas de software tradicionales se resuelve con bloqueos (locks): antes de modificar un dato compartido, el proceso adquiere un bloqueo exclusivo sobre ese dato y lo libera cuando termina. Otros procesos que quieren modificar el mismo dato deben esperar a que el bloqueo sea liberado.

Los bloqueos funcionan pero tienen costos: en un sistema multiagente donde los agentes pueden tardar segundos o decenas de segundos en completar una tarea, un bloqueo mantenido durante toda la ejecución de un agente puede bloquear a otros agentes por períodos inaceptablemente largos.

Tres estrategias de memoria compartida

Estrategia 1: Almacén de estado centralizado con escrituras atómicas

El almacén de estado centralizado serializa todas las escrituras. Cuando un agente quiere escribir al estado, su escritura se ejecuta de forma atómica: se aplica completamente o no se aplica. Si dos escrituras llegan simultáneamente, se ejecutan en serie, una después de la otra, sin posibilidad de solapamiento parcial.

Esta estrategia elimina las condiciones de carrera pero introduce un cuello de botella: el almacén de estado centralizado procesa las escrituras en serie. En sistemas con pocos agentes y escrituras infrecuentes, esto no es un problema. En sistemas con muchos agentes y alto volumen de escrituras, puede convertirse en el limitante de rendimiento del sistema.

Estrategia 2: Estado versionado con control optimista

Cada objeto en el estado tiene un número de versión. Cuando un agente quiere modificar un objeto, lee el objeto y toma nota de su versión actual. Produce su modificación. Cuando intenta escribir, el sistema verifica que la versión del objeto no cambió desde que el agente lo leyó. Si la versión es la misma, la escritura se acepta y la versión se incrementa. Si la versión cambió (otro agente modificó el objeto en el intervalo), la escritura se rechaza y el agente debe leer la versión actualizada y reintentar.

Este mecanismo de control de concurrencia optimista es el modelo correcto cuando los conflictos son poco frecuentes. Cuando son frecuentes —cuando muchos agentes intentan modificar el mismo objeto al mismo tiempo— el costo de los reintentos puede ser mayor que el de los bloqueos. La elección depende del patrón de acceso específico del sistema.

Estrategia 3: Estado distribuido mediante paso de mensajes

En lugar de un almacén centralizado, cada agente mantiene su propio estado local. La sincronización entre agentes ocurre mediante mensajes: cuando un agente produce información relevante para otros agentes, la publica como un mensaje. Los agentes que necesitan esa información la reciben y actualizan su estado local.

Este modelo elimina el problema de concurrencia sobre un estado centralizado porque no hay estado centralizado. Cada agente tiene su propia copia de la información que necesita. El costo es la consistencia eventual: en el momento en que un agente publica una actualización, otros agentes pueden no haberla recibido todavía. Hay un período durante el cual diferentes agentes tienen visiones distintas del estado del sistema. Para sistemas donde la consistencia inmediata es crítica, este modelo no es apropiado. Para sistemas donde cada agente trabaja principalmente con su propio contexto y solo necesita sincronización periódica, el modelo es muy eficiente.

Qué guardar en la memoria compartida

No toda la información que un agente produce debe ir al estado compartido. Parte de la información es interna al agente y no necesita ser visible para otros. Parte es relevante solo para el próximo agente en la cadena. Parte es relevante para todo el sistema durante toda la duración de la tarea.

Un criterio útil para decidir qué va al estado compartido:

Compartido y persistente: información que múltiples agentes necesitan consultar, y que debe estar disponible durante toda la duración de la tarea. El resultado de investigaciones, conclusiones validadas por el supervisor, el estado de progreso del plan. Esta información va al estado compartido.

Compartido y efímero: información que un agente necesita transferir a otro agente específico, pero que no es relevante para el sistema en general. Este tipo de información se transmite mejor como un mensaje directo entre agentes que como estado compartido.

Local y efímero: el razonamiento interno del agente durante la ejecución de su tarea, los resultados intermedios que descarta, los intentos fallidos antes del output final. Esta información no debe ir al estado compartido. Incluirla añade ruido y costo sin valor.

El contexto que cada agente recibe

Una pregunta práctica frecuente es: cuando un agente comienza a trabajar en su subtarea, ¿qué información del estado compartido debe recibir?

La respuesta correcta no es "todo el estado": un agente que recibe todo el contexto acumulado del sistema puede estar procesando información irrelevante para su subtask específica, lo que añade costo de tokens y puede distraer su razonamiento. La respuesta correcta es: el agente recibe el contexto mínimo necesario para ejecutar su subtask más los resultados de las subtasks de las cuales la suya depende.

El orquestador o planificador que asigna la subtask al agente es responsable de construir este contexto mínimo necesario. No delega al agente la responsabilidad de decidir qué del estado global necesita: lo construye explícitamente como parte del mensaje de tarea que le envía al agente.

Privacidad y aislamiento en la memoria compartida

En sistemas multiagente empresariales, el estado compartido puede contener información sensible. Una arquitectura que permite que cualquier agente lea cualquier parte del estado es un riesgo de privacidad: un agente comprometido, o un agente con un error de razonamiento que lo lleva a incluir información inapropiada en su output, puede exponer información a la que no debería tener acceso.

El principio de mínimo privilegio aplica también al acceso al estado: cada agente debe tener acceso de lectura solo a las partes del estado que su función requiere, y acceso de escritura solo a las partes del estado que su función produce. El control de acceso al estado compartido no es una funcionalidad opcional en producción: es una condición necesaria para que el sistema sea seguro.

La sección 08 amplía la perspectiva desde el estado interno del sistema hacia sus propiedades operacionales: cómo se escala cuando la demanda crece, y cómo se comporta cuando uno o más agentes fallan. La tolerancia a fallos en sistemas multiagente no es un tema de mantenimiento: es un tema de diseño que debe resolverse antes de la primera línea de código de producción.

Capítulo 09 — Arquitecturas Multiagente

Sección 08 — Escalabilidad y tolerancia a fallos

Un sistema multiagente que funciona perfectamente en un entorno de desarrollo controlado y falla en producción bajo carga real no es un sistema terminado. Es un prototipo. La diferencia entre ambos no está en la lógica del negocio ni en la calidad del razonamiento de los agentes: está en la ingeniería de las propiedades operacionales del sistema. La escalabilidad define qué ocurre cuando la demanda crece. La tolerancia a fallos define qué ocurre cuando algo sale mal.

Ambas propiedades deben diseñarse antes de la primera línea de código de producción. Añadir las como correcciones posteriores a un sistema ya en producción es costoso, lento y frecuentemente incompleto.

Escalabilidad horizontal de agentes

La forma natural de escalar un sistema multiagente es la escalabilidad horizontal: añadir más instancias de los agentes que constituyen el cuello de botella. Si el sistema procesa cien solicitudes por hora y el agente analista tarda treinta segundos por solicitud, ese agente es el limitante. Añadir una segunda instancia del agente analista duplica la capacidad de ese paso sin modificar ninguna otra parte del sistema.

Esta escalabilidad horizontal funciona cuando los agentes son stateless: cuando su comportamiento depende únicamente del mensaje que recibieron, no de ningún estado interno acumulado de interacciones anteriores. Un agente stateless puede escalar a cualquier número de instancias porque todas las instancias son equivalentes entre sí.

Cuando los agentes tienen estado interno —cuando una instancia del agente analista necesita recordar el contexto de solicitudes anteriores para funcionar correctamente—, la escalabilidad horizontal se complica. Las solicitudes relacionadas deben dirigirse a la misma instancia del agente (afinidad de sesión), lo que limita la flexibilidad de la distribución de carga.

El principio de diseño que emerge: diseña los agentes como stateless siempre que sea posible. Si el agente necesita estado, externaliza ese estado al almacén compartido que la sección 07

describió. La instancia del agente lee el estado del almacén al inicio de cada tarea y escribe sus resultados al almacén al finalizar. El agente en sí mismo no guarda nada entre tareas.

Costo y latencia del paralelismo

El paralelismo de agentes no es gratis. Cada agente en ejecución simultánea consume tokens de input y output en paralelo. Un sistema que ejecuta diez agentes en paralelo puede consumir diez veces el costo de tokens de un agente único ejecutando en serie, incluso si la latencia total es menor.

Esta relación entre paralelismo, costo y latencia tiene implicaciones concretas para el diseño:

El paralelismo reduce la latencia pero aumenta el costo. Cuando la latencia es la variable crítica —el usuario está esperando una respuesta en tiempo real—, el paralelismo es la herramienta correcta. Cuando el costo es la variable crítica —el sistema procesa miles de solicitudes por hora en un presupuesto definido—, el paralelismo excesivo puede no estar justificado.

El grado óptimo de paralelismo depende del problema. No todas las subtareas de un sistema se benefician de la misma cantidad de paralelismo. Las subtareas largas y costosas son las mejores candidatas para el paralelismo. Las subtareas cortas pueden no justificar el overhead de coordinación.

El costo de los agentes de coordinación se suma. El orquestador, el planificador y el supervisor también consumen tokens. En un sistema con muchos niveles de coordinación, el costo de los agentes de coordinación puede ser una fracción significativa del costo total. Este costo debe estar en el modelo de costo del sistema desde el diseño.

Modos de fallo en sistemas multiagente

Los fallos en sistemas multiagente pueden clasificarse en cuatro categorías según su naturaleza y sus implicaciones:

Fallo de un agente ejecutor. El agente que debía completar una subtarea no puede hacerlo: el servicio de IA no está disponible, el timeout se agotó, el agente produjo un error irreparable. Este es el fallo más común y el que el sistema debe manejar con más frecuencia.

Fallo del orquestador o planificador. El agente de coordinación del sistema falla. Este es el fallo más severo en una topología jerárquica: sin el orquestador, el sistema no puede coordinar trabajo. En una topología entre pares, el fallo de un agente individual tiene un impacto más localizado.

Fallo silencioso. El agente completa la tarea sin errores técnicos pero produce un output incorrecto o de baja calidad que el sistema acepta como válido. Este tipo de fallo es el más peligroso porque no genera alertas: el sistema funciona correctamente desde el punto de vista técnico, pero el resultado entregado al usuario es incorrecto. El supervisor de la sección 06 es la defensa principal contra este tipo de fallo.

Fallo de la memoria compartida. El almacén de estado no está disponible o devuelve datos corruptos. Los agentes que dependen de ese estado no pueden completar su trabajo. Este fallo es transversal a todo el sistema: no afecta a un agente sino a todos.

Estrategias de resiliencia

Reintentos con backoff exponencial. Cuando un agente ejecutor falla en una subtarea, el sistema reintentla la subtarea después de un intervalo de espera. Si el segundo intento también falla, el intervalo se duplica antes del siguiente reintent. Este patrón da tiempo al sistema de IA subyacente para recuperarse de condiciones transitorias sin generar una cascada de solicitudes que puede empeorar el problema.

El límite de reintentos es un parámetro de diseño importante. Un límite demasiado bajo puede abandonar prematuramente una tarea que se habría recuperado. Un límite demasiado alto puede mantener bloqueado a un agente esperando la recuperación de un servicio que no va a recuperarse a tiempo.

Agente de respaldo (fallback agent). Para subtarear críticas, se puede designar un agente alternativo que recibe la tarea si el agente primario no puede completarla. El agente de respaldo puede ser una instancia del mismo tipo de agente en un entorno diferente, un agente con capacidades reducidas pero disponibilidad más alta, o un comportamiento degradado que produce un resultado parcial en lugar de ningún resultado.

Circuit breaker. Si un agente falla repetidamente dentro de un período definido, el sistema lo marca como no disponible y deja de enviarle solicitudes por un tiempo determinado. Este patrón evita que el sistema continúe enviando solicitudes a un componente que claramente no puede procesarlas, permitiendo que el componente se recupere sin estar inundado de solicitudes nuevas.

Punto de verificación y recuperación (checkpoint and recovery). En tareas de larga duración, el sistema guarda el estado del progreso en puntos intermedios. Si el sistema falla después del tercer paso de un plan de ocho, la recuperación puede retomar desde el tercer paso en lugar de reiniciar desde el primero. Esta estrategia es especialmente valiosa en

sistemas donde el costo de reiniciar desde cero es alto, ya sea en tiempo, en tokens o en acceso a recursos externos que no son gratuitos.

Observabilidad como condición de operación

Un sistema multiagente sin observabilidad es un sistema que no se puede operar. Cuando algo falla —o cuando algo funciona peor de lo esperado—, la capacidad de determinar qué falló, cuándo y por qué depende de la instrumentación del sistema.

La observabilidad mínima en un sistema multiagente incluye:

- Trazas de cada mensaje enviado y recibido entre agentes, con marcas temporales.
- Logs de cada subtask: cuándo comenzó, qué input recibió, qué output produjo, cuánto tiempo tomó, si requirió reintentos.
- Métricas de costo por agente y por tarea, para detectar desviaciones respecto a los valores esperados.
- Alertas sobre fallos recurrentes de agentes específicos o sobre tasas de error que exceden umbrales predefinidos.

Sin esta instrumentación, un sistema en producción es una caja negra. Con ella, el comportamiento del sistema es visible y las anomalías son detectables antes de que se conviertan en problemas que los usuarios reportan.

La sección 09 toma todo lo desarrollado en las secciones anteriores y lo organiza en el lenguaje de los patrones: qué configuraciones de diseño funcionan de forma consistente en sistemas multiagente reales, y qué configuraciones producen problemas igualmente consistentes. Conocer los patrones y los anti-patrones es lo que permite reconocerlos en un sistema propio.

Capítulo 09 — Arquitecturas Multiagente

Sección 09 — Patrones y anti-patrones multiagente

Los patrones de diseño en ingeniería de software son soluciones probadas a problemas recurrentes. No son recetas que se aplican mecánicamente: son marcos que nombran una solución conocida para que los equipos puedan reconocerla, comunicarla y aplicarla con conciencia de sus compromisos. Los anti-patrones son su inverso: configuraciones que parecen soluciones pero que producen problemas específicos y conocidos.

En el diseño de sistemas multiagente, el vocabulario de patrones y anti-patrones está consolidándose a medida que más equipos ponen estos sistemas en producción. Esta sección presenta los patrones y anti-patrones más relevantes, no como categorías exhaustivas sino como el conjunto que un AI Engineer debe reconocer antes de diseñar su primer sistema multiagente.

Patrones que funcionan

Patrón: Reflexión (Reflection)

Un agente generador produce un output. Ese output es enviado a un agente crítico que lo evalúa contra criterios explícitos y produce una evaluación. Si la evaluación identifica problemas, el output vuelve al agente generador con el feedback específico para que produzca una versión mejorada. El ciclo puede repetirse hasta que el output pase la evaluación o hasta que se alcance el límite de iteraciones.

El patrón de Reflexión es especialmente valioso para tareas donde la precisión del output es alta y donde un agente único tiene dificultades para detectar sus propios errores. En código generado por IA, la Reflexión puede detectar errores que el agente generador validó como correctos. En análisis complejos, puede identificar saltos lógicos que el agente analista no advirtió.

El costo del patrón es la latencia adicional y el costo de tokens del agente crítico. El beneficio es una tasa de error significativamente menor en el output final. La decisión de aplicarlo depende de si la criticidad de la tarea justifica ese costo adicional.

Patrón: Ejecución paralela con síntesis (Parallel Fanout with Synthesis)

Un agente orquestador descompone una tarea en subtareas independientes, las distribuye simultáneamente a múltiples agentes especializados y espera a que todos completen su trabajo. Una vez que todos los agentes han reportado sus resultados, el orquestador (o un agente síntesis dedicado) integra esos resultados en un output unificado.

Este patrón es el más directo para aprovechar la arquitectura multiagente: la latencia total es la del agente más lento, no la suma de todas las latencias. Es el patrón correcto cuando las subtareas son genuinamente independientes y el resultado final requiere la integración de todos los outputs parciales.

El riesgo más frecuente es asumir independencia entre subtareas cuando en realidad hay dependencias sutiles. Un sistema que analiza en paralelo los aspectos financieros y legales de una empresa puede producir conclusiones que se contradicen si los datos que cada agente analiza no son consistentes entre sí. La independencia de las subtareas es una condición que debe verificarse, no asumirse.

Patrón: Escalada supervisada (Supervised Escalation)

El sistema tiene un agente de nivel básico que maneja la mayoría de los casos, y un agente de nivel superior —más capaz, más costoso— que recibe los casos que el agente básico no pudo resolver con confianza. La decisión de escalar puede basarse en una métrica de confianza del agente básico, en la detección de características del caso que lo clasifican como complejo, o en la intervención del supervisor.

Este patrón reduce el costo promedio del sistema porque la mayoría de los casos son resueltos por el agente de menor costo. El agente de nivel superior solo se activa cuando es genuinamente necesario. La eficiencia del patrón depende de la capacidad del sistema para identificar correctamente qué casos requieren escalada: si muchos casos se escalan innecesariamente, el beneficio de costo se pierde.

Patrón: Verificación cruzada (Cross-Validation)

Dos o más agentes resuelven independientemente el mismo problema, y un agente árbitro compara sus resultados. Si los resultados coinciden, el consenso se acepta como la respuesta del sistema. Si hay discrepancias significativas, el árbitro puede intentar resolver el conflicto mediante análisis adicional o escalar a revisión humana.

Este patrón duplica o triplica el costo de la generación pero ofrece el nivel más alto de confianza en el output para tareas de máxima criticidad. Es la arquitectura correcta cuando el costo de un error en el output supera con creces el costo adicional de la generación redundante.

Anti-patrones que se deben evitar

Anti-patrón: El agente dios (God Agent)

Un único agente que intenta hacer todo: buscar información, analizarla, redactar el output, verificarlo, ejecutar acciones y coordinar a otros agentes. El resultado es un agente con un prompt de sistema extremadamente largo, herramientas de naturaleza muy distinta y un comportamiento difícil de predecir porque está intentando equilibrar demasiadas responsabilidades simultáneamente.

El síntoma más claro del anti-patrón del agente dios es el prompt de sistema que empieza con "Eres un asistente inteligente capaz de..." seguido de una lista de decenas de capacidades heterogéneas. La solución es la descomposición: identificar las responsabilidades del agente dios y redistribuirlas entre agentes especializados con roles bien delimitados.

Anti-patrón: Red de charlas (Chatty Network)

Los agentes se comunican entre sí con una frecuencia mucho mayor de lo necesario. En lugar de que un agente complete una subtarea completa y reporte el resultado, el agente A envía al agente B un resultado parcial, B responde con una consulta, A responde a la consulta, B envía un resultado parcial a C, C responde a B... El sistema produce un volumen de comunicación que excede con creces la complejidad del trabajo real que está haciendo.

Este anti-patrón generalmente emerge cuando los roles de los agentes no están bien definidos (cada agente necesita consultar constantemente a otros porque no tiene claro qué información necesita para completar su tarea) o cuando el sistema fue diseñado con demasiado acoplamiento entre agentes (el output del agente A tiene demasiadas dependencias del estado del agente B).

La solución es revisar la definición de roles y asegurarse de que cada agente recibe, al inicio de su tarea, toda la información que necesita para completarla sin consultas adicionales.

Anti-patrón: Confianza no verificada entre agentes (Unverified Trust)

Un agente acepta el output de otro agente como verdadero sin verificación. Si el agente upstream produce un error —ya sea por un fallo de razonamiento o por manipulación externa

— ese error se propaga a través del sistema sin ningún punto de detección.

En sistemas multiagente, los agentes deben tratar el output de otros agentes con el mismo escepticismo que aplicarían a datos de fuentes externas. La validación del esquema del output (¿el formato es el esperado?), la verificación de rangos plausibles (¿los valores están en rangos razonables?) y la detección de inconsistencias internas (¿las partes del output son coherentes entre sí?) son controles que deben estar presentes aunque el origen del dato sea otro agente del mismo sistema.

Anti-patrón: Multiagente innecesario (Accidental Complexity)

El sistema implementa una arquitectura multiagente para resolver un problema que un agente único bien diseñado podría resolver con mejor calidad, menor costo y mayor simplicidad. Este anti-patrón no tiene un síntoma técnico inmediato: el sistema funciona. El síntoma es la dificultad desproporcionada para mantener, depurar y mejorar el sistema en comparación con el valor que produce.

La prueba de diagnóstico es la pregunta de las cuatro condiciones de la sección 02: si ninguna de ellas está presente (paralelismo genuinamente necesario, especialización fundamentalmente distinta, criticidad que requiere verificación independiente, volumen que excede la ventana de contexto), la arquitectura multiagente es accidental, no necesaria.

Anti-patrón: Plan sin supervisión (Unmonitored Plan)

El planificador genera un plan, los agentes lo ejecutan, y el resultado final se entrega al usuario sin que ningún punto del proceso incluya verificación de calidad. El sistema puede operar durante días entregando outputs incorrectos sin que nadie lo detecte, porque no hay ningún mecanismo de detección.

Este anti-patrón suele emerger en la transición del prototipo a la producción: en el prototipo, el desarrollador revisa manualmente cada output. En producción, esa revisión manual ya no es posible a escala, pero tampoco se implementó ningún sustituto automatizado. La observabilidad de la sección 08 y el patrón de supervisión de la sección 06 son las respuestas a este anti-patrón.

La sección 10 aplica los patrones y principios de este capítulo a un caso de estudio concreto. El salto de la teoría a la práctica requiere ver un sistema real diseñado paso a paso, con las decisiones que se tomaron y las alternativas que se descartaron.

Capítulo 09 — Arquitecturas Multiagente

Sección 10 — Caso de estudio empresarial

La teoría de los sistemas multiagente adquiere claridad cuando se aplica a un caso concreto. Esta sección desarrolla el diseño completo de un sistema multiagente para un problema empresarial real: un sistema de análisis de propuestas de proveedores para el departamento de compras de una empresa de manufactura mediana.

El caso no es trivial ni es artificialmente complejo. Fue elegido porque tiene características que justifican genuinamente una arquitectura multiagente, y porque las decisiones de diseño que requiere son las mismas que aparecen en la mayoría de los sistemas empresariales de esta clase.

El problema

La empresa recibe entre veinte y cincuenta propuestas de proveedores por mes para distintas categorías de compra: materiales, equipamiento, servicios de logística, tecnología. Cada propuesta incluye documentación técnica, condiciones comerciales, referencias de clientes y, en algunos casos, muestras o demos.

El equipo de compras actualmente revisa estas propuestas manualmente. El proceso toma entre dos y cinco días hábiles por propuesta, dependiendo de la complejidad. El resultado es un informe de evaluación que compara al proveedor contra los criterios de la empresa en cada dimensión relevante y produce una recomendación: aprobar, rechazar o solicitar información adicional.

El equipo de compras quiere un sistema que produzca borradores de evaluación de alta calidad en horas, no en días. Un analista humano revisará cada borrador antes de que sea comunicado externamente, por lo que el sistema no necesita operar sin supervisión humana. Pero la calidad del borrador debe ser suficientemente alta como para que la revisión humana sea un proceso de verificación, no una reescritura.

Análisis de la arquitectura

¿Justifica este problema una arquitectura multiagente? Aplicando las cuatro preguntas de la sección 02:

- ¿Subtareas independientes paralelizables? Sí. El análisis técnico, el análisis comercial, la verificación de referencias y la búsqueda de información pública sobre el proveedor son independientes entre sí. Un agente puede analizar las condiciones comerciales mientras otro verifica las referencias simultáneamente.
- ¿Especialización fundamentalmente distinta por dominio? Sí. El análisis técnico requiere conocimiento del dominio de manufactura y los criterios técnicos de la empresa. El análisis comercial requiere conocimiento de condiciones contractuales estándar y parámetros financieros. Son dominios suficientemente distintos para justificar agentes especializados.
- ¿Criticidad que justifica verificación independiente? Sí. Una evaluación incorrecta puede llevar a la empresa a contratar un proveedor inadecuado, con impacto en costos, calidad de producción o continuidad operacional. La calidad del output justifica supervisión.
- ¿Volumen que excede la ventana de contexto? Depende de la propuesta. Propuestas complejas con documentación extensa pueden exceder la ventana de un agente único. La arquitectura multiagente resuelve también este problema.

La respuesta es sí en todas las dimensiones relevantes. La arquitectura multiagente está justificada.

Diseño del sistema

Agentes del sistema:

El sistema tiene cinco agentes con roles claramente delimitados:

Agente orquestador: recibe la propuesta, extrae los metadatos básicos (proveedor, categoría de compra, fecha), lanza el plan de análisis en paralelo y coordina la síntesis final. No produce análisis propio.

Agente de análisis técnico: recibe la documentación técnica de la propuesta y los criterios técnicos de la empresa para esa categoría de compra. Evalúa si las especificaciones del proveedor cumplen los requisitos técnicos mínimos y los diferenciadores positivos. Produce un informe estructurado con evaluación por dimensión técnica.

Agente de análisis comercial: recibe las condiciones comerciales de la propuesta: precios, plazos de pago, penalidades, cláusulas de ajuste, garantías. Las evalúa contra los parámetros

comerciales estándar de la empresa. Produce un informe con el análisis de riesgo comercial y las condiciones que requieren negociación.

Agente de investigación de proveedor: realiza búsquedas sobre la reputación pública del proveedor: cobertura mediática, litigios conocidos, certificaciones verificadas, presencia en mercados relevantes. Produce un resumen de hallazgos con fuentes.

Agente redactor: recibe los tres informes de los agentes especializados y el resumen de investigación. Redacta el borrador de evaluación en el formato estándar de la empresa, integrando los hallazgos de cada dimensión con coherencia y sin contradicciones. Produce el documento final para revisión humana.

Topología: jerárquica. El orquestador coordina el proceso. Los tres agentes especializados trabajan en paralelo. El redactor espera a tener todos los informes antes de comenzar.

Mecanismo de comunicación: mensajes estructurados en JSON. El orquestador envía a cada agente especializado un mensaje con el tipo de análisis requerido y los documentos relevantes como payload. Los agentes especializados responden con informes estructurados en JSON. El redactor recibe los tres informes y produce el texto final.

Estado compartido: el sistema mantiene un estado de progreso por propuesta que incluye: estado de cada análisis (pendiente, en curso, completado, con error), los informes producidos por cada agente y el borrador final cuando esté disponible. El orquestador actualiza este estado a medida que cada agente reporta.

Supervisión: el sistema no incluye un agente supervisor automático porque la revisión humana actúa como supervisor final. Lo que el sistema sí incluye es validación de esquema en cada informe: si un agente produce un informe que no cumple el esquema esperado, el sistema lo rechaza y reintenta la tarea antes de continuar.

Decisiones de diseño y alternativas descartadas

¿Por qué no hay un agente supervisor automático? Porque el costo adicional y la latencia de un ciclo de supervisión automático no están justificados cuando hay un revisor humano como paso final obligatorio. Si la empresa quisiera que el sistema operara sin revisión humana —lo que en este contexto no es el caso— añadir supervisión automática sería la decisión correcta.

¿Por qué el redactor es un agente separado y no el orquestador? Porque la redacción de un documento de evaluación es una tarea especializada: requiere un prompt de

sistema orientado a la síntesis, el formato estándar de la empresa y el tono apropiado para comunicación interna de compras. El orquestador tiene un rol de coordinación, no de redacción. Mezclar ambos roles en un único agente degradaría la calidad de ambas funciones.

¿Por qué el análisis técnico y el análisis comercial son agentes separados? Porque sus instrucciones de sistema, sus herramientas y sus criterios de evaluación son fundamentalmente distintos. Un analista técnico que evalúa especificaciones de materiales no tiene el mismo marco de referencia que un analista comercial que evalúa condiciones contractuales. Separar los roles permite optimizar cada agente para su función específica.

Estimación de costo y latencia

Para una propuesta de complejidad media, el sistema estima:

- Análisis paralelo (técnico + comercial + investigación): 45 a 90 segundos en paralelo.
- Redacción del borrador: 20 a 40 segundos.
- Tiempo total por propuesta: 65 a 130 segundos.
- Costo de tokens estimado por propuesta: equivalente a tres o cuatro llamadas a un modelo de alta capacidad (dado el contexto de cada agente).

Este costo es significativamente menor que el costo del tiempo humano de dos a cinco días de análisis. La calidad del borrador producido, según validación con el equipo de compras, reduce el tiempo de revisión humana de dos a cinco días a una o dos horas de verificación y ajustes.

La sección 11 convierte este caso en un ejercicio práctico. El lector tomará un escenario diferente y diseñará el sistema multiagente desde cero, aplicando los criterios, patrones y decisiones de diseño desarrollados a lo largo del capítulo.

Capítulo 09 — Arquitecturas Multiagente

Sección 11 — Laboratorio práctico

Este laboratorio es un ejercicio de diseño arquitectónico. No requiere código. Requiere aplicar los criterios, patrones y principios desarrollados en las secciones anteriores para diseñar un sistema multiagente completo para un caso de negocio real.

El objetivo del laboratorio no es producir el diseño perfecto en el primer intento. Es desarrollar el proceso de pensamiento que un AI Engineer necesita para abordar sistemáticamente el diseño de un sistema multiagente: hacer las preguntas correctas, en el orden correcto, y tomar decisiones explícitas con sus fundamentos.

El escenario

Una empresa de medios digitales publica artículos de análisis económico y financiero para una audiencia de ejecutivos y analistas. El equipo editorial actualmente produce doce artículos por semana, con un proceso que incluye investigación de fuentes, análisis de datos, redacción, verificación de hechos y revisión editorial.

El director editorial quiere un sistema de IA que pueda producir borradores de artículos a partir de un brief editorial breve. El brief especifica el tema, la audiencia objetivo, los puntos principales que el artículo debe cubrir, las fuentes que el equipo ya identificó como relevantes y la extensión aproximada.

El output esperado es un borrador de artículo completo, estructurado, con datos verificados y fuentes citadas, listo para revisión editorial humana. La revisión humana es obligatoria: el sistema no publicará directamente. Pero la calidad del borrador debe ser suficientemente alta para que la revisión sea un proceso de refinamiento, no de reescritura.

El equipo editorial producirá un mínimo de tres briefs por día hábil. Cada artículo tiene entre 800 y 1.500 palabras.

Estructura del ejercicio

El estudiante debe producir un documento de diseño que incluya los siguientes elementos, en orden:

1. Análisis de justificación

Responde las cuatro preguntas de la sección 02 para este escenario específico:

- ¿Hay subtarefas independientes que se benefician del paralelismo? ¿Cuáles son?
- ¿Los distintos aspectos de la tarea requieren especialización fundamentalmente distinta?
¿En qué dominios?
- ¿La criticidad del output justifica verificación independiente? ¿Por qué sí o por qué no?
- ¿El volumen de información puede exceder la ventana de contexto de un agente único?
¿En qué condiciones?

La respuesta a cada pregunta debe incluir el razonamiento, no solo la conclusión.

2. Definición de agentes

Para cada agente del sistema, especifica:

- Nombre del agente y su rol funcional.
- Alcance funcional: qué hace y qué no hace.
- Herramientas que necesita (búsqueda web, acceso a bases de datos específicas, herramientas de cómputo, etc.).
- Un párrafo que describe su instrucción de sistema en términos generales: qué objetivos tiene, qué restricciones aplican, qué formato de output produce.

El número de agentes no está predefinido. El estudiante decide cuántos son necesarios con base en el análisis de la tarea.

3. Topología del sistema

Dibuja (o describe en texto con suficiente detalle para que pueda dibujarse) el diagrama de topología del sistema:

- Cada agente, representado como un nodo con su nombre y rol.
- Las conexiones entre agentes, con la dirección del flujo de información (quién le envía qué a quién).
- El orden de ejecución: qué agentes actúan en paralelo, qué agentes actúan en secuencia, qué dependencias existen entre ellos.

4. Protocolo de comunicación

Para las tres conexiones más importantes del sistema (las que tienen mayor impacto en la calidad o el rendimiento):

- El formato del mensaje que fluye por esa conexión (descripción del esquema JSON o del formato elegido).
- Si la comunicación es síncrona o asíncrona, y por qué.
- Qué debe hacer el receptor si el mensaje llega malformado o si el emisor no responde dentro del tiempo esperado.

5. Estrategia de memoria compartida

Describe el almacén de estado del sistema:

- Qué información se guarda en el estado compartido.
- Qué información permanece local a cada agente.
- Cómo se resuelven los conflictos de escritura simultánea (si existen en tu diseño).
- Qué información persiste después de que la tarea termina y cuál se descarta.

6. Manejo de fallos

Para cada uno de estos escenarios de fallo, describe qué debe hacer el sistema:

- El agente de búsqueda de fuentes no encuentra ninguna fuente relevante.
- El agente verificador de datos detecta que una afirmación del borrador no puede ser verificada.
- El agente redactor produce un output que no cumple el esquema de formato esperado.
- El sistema recibió dos briefs simultáneamente y los recursos están al límite.

7. Estimación de costo y latencia

Estima, con los supuestos que consideres razonables y que debes explicitar:

- Tiempo total esperado para producir un borrador de 1.200 palabras.
- Número aproximado de llamadas al modelo de lenguaje que realiza el sistema para esa tarea.
- Los dos factores que más influyen en el costo del sistema y cómo podrían optimizarse si el presupuesto fuera restrictivo.

Criterios de evaluación del diseño

Un buen diseño en este laboratorio tiene las siguientes características:

- Los roles de los agentes son claros, no solapados y necesarios: no sobran agentes ni faltan.
- La topología elegida es coherente con la naturaleza de las subtarear (paralelas o secuenciales).
- Los protocolos de comunicación están definidos con suficiente precisión para ser implementables sin ambigüedad.
- El manejo de fallos es explícito y cubre los escenarios más probables, no solo el caso nominal.
- Las decisiones de diseño tienen fundamentos articulados: cuando hay varias opciones posibles, el diseño elige una y explica por qué.

Un diseño que cumple estas características demuestra que el estudiante puede tomar las decisiones arquitectónicas que un sistema multiagente real requiere. No demuestra que ese sea el único diseño correcto: hay múltiples diseños válidos para el mismo problema. Lo que distingue un buen diseño es la coherencia entre el problema, las decisiones tomadas y los fundamentos que las sustentan.

Variante de extensión

Para quienes quieran profundizar el ejercicio, la variante de extensión plantea el siguiente desafío adicional:

El director editorial quiere que el sistema pueda aceptar un artículo publicado anteriormente por un competidor como referencia de estilo y tono. El sistema debe producir borradores que se acerquen al estilo del artículo de referencia sin copiar su contenido.

¿Cómo modifica esta nueva funcionalidad el diseño del sistema? ¿Requiere un agente adicional? ¿Cambia la instrucción de sistema del agente redactor? ¿Qué parte del estado compartido necesita actualizarse?

La sección 12 provee la checklist operativa que consolida los criterios de decisión, los requisitos de diseño y las verificaciones de calidad de este capítulo en un formato accionable para el trabajo diario del AI Engineer.

Capítulo 09 — Arquitecturas Multiagente

Sección 12 — Checklist del AI Engineer

Esta checklist consolida los criterios y verificaciones de diseño del capítulo en un formato accionable. Está organizada en tres momentos del proceso: antes de diseñar el sistema (decisión de arquitectura), durante el diseño (calidad del diseño), y antes de pasar a producción (verificación operacional).

No es una lista de control mecánica. Cada ítem es una pregunta que requiere una respuesta genuina, no una casilla para marcar. Si no puedes responder una pregunta con precisión, esa es la señal de que esa dimensión del sistema necesita más trabajo antes de avanzar.

Bloque 1: Decisión de arquitectura

Completa este bloque antes de comprometerte con una arquitectura multiagente. Si ninguno de los primeros cuatro ítems produce una respuesta afirmativa, considera si un agente único bien diseñado resuelve el problema de forma más simple.

- [] **Paralelismo justificado.** La tarea tiene subtarefas independientes que se benefician de ejecución simultánea. Puedo nombrar cuáles son y demostrar que son genuinamente independientes.
- [] **Especialización justificada.** Las distintas dimensiones de la tarea requieren conocimiento, herramientas o instrucciones de sistema fundamentalmente distintos. Puedo describir en qué difieren los dominios de trabajo de cada agente previsto.
- [] **Verificación independiente justificada.** La criticidad del output justifica que un segundo agente evalúe el trabajo del primero antes de que sea aceptado. Puedo articular qué consecuencias tiene un error en el output.
- [] **Distribución de contexto justificada.** El volumen de información que la tarea requiere procesar excede lo que puede mantenerse en una sola ventana de contexto con calidad aceptable.

- [] **Agente único descartado explícitamente.** Intenté resolver el problema con un agente único y encontré límites específicos. Puedo describir cuáles son esos límites.
-

Bloque 2: Diseño de los agentes

Completa este bloque para cada agente del sistema antes de implementarlo.

- [] **Rol limitado.** El agente tiene una función específica y delimitada. Puedo describirla en una oración sin usar "y también".
 - [] **Instrucción de sistema coherente con el rol.** La instrucción de sistema expresa el rol con precisión: incluye el objetivo, las restricciones y el formato de output esperado. No incluye capacidades que el agente no necesita para su función.
 - [] **Herramientas mínimas necesarias.** El agente tiene acceso solo a las herramientas que su función específicamente requiere. Puedo justificar por qué cada herramienta incluida es necesaria.
 - [] **Formato de output definido.** El output que este agente produce tiene un esquema definido, conocido por el agente que lo consumirá. No hay ambigüedad sobre qué estructura tendrá el resultado.
 - [] **Sin estado interno entre tareas.** El agente opera sobre el input que recibe y el estado compartido al que tiene acceso. No acumula estado interno que dependa de interacciones anteriores. Si necesita información de contexto anterior, está disponible en el estado compartido.
-

Bloque 3: Diseño del sistema

Completa este bloque para el sistema en su conjunto.

- [] **Topología documentada.** Existe un diagrama del sistema que muestra todos los agentes, sus conexiones, la dirección del flujo de información y el orden de ejecución (paralelo o secuencial para cada parte del sistema).
- [] **Dependencias explícitas.** Las dependencias entre agentes (el agente B requiere el output del agente A) están explicitadas en el diagrama y en la estructura del plan de ejecución.

- [] **Protocolos de mensajes definidos.** Para cada conexión entre agentes existe un esquema definido para el mensaje. Los tipos de mensaje (tarea, resultado, error) están especificados.
 - [] **Supervisión diseñada.** El sistema tiene un mecanismo para detectar cuando un agente produce un output de calidad inaceptable. Ese mecanismo puede ser automático (agente supervisor) o manual (revisión humana), pero está diseñado explícitamente, no asumido implícitamente.
 - [] **Sin roles solapados.** No hay dos agentes que hacen el mismo trabajo ni vacíos donde ningún agente es responsable de una parte necesaria de la tarea.
-

Bloque 4: Memoria compartida y estado

- [] **Almacén de estado diseñado.** Existe una definición del almacén de estado compartido: qué campos contiene, quién puede leer cada campo, quién puede escribir cada campo.
 - [] **Conflictos de escritura resueltos.** Si dos agentes pueden escribir en el mismo campo simultáneamente, hay un mecanismo para resolver ese conflicto (serialización, versionamiento con control optimista, o separación de escrituras por diseño).
 - [] **Contexto mínimo por agente.** Cada agente recibe el contexto mínimo necesario para su subtarea, no el estado completo del sistema. El orquestador construye ese contexto mínimo como parte del mensaje de tarea.
 - [] **Privacidad y aislamiento.** Cada agente tiene acceso de lectura y escritura solo a las partes del estado que su función requiere. No hay acceso generalizado a todo el estado para todos los agentes.
-

Bloque 5: Resiliencia operacional

- [] **Fallos de agentes individuales manejados.** Cada agente tiene una estrategia de manejo de error: reintentos con backoff, agente de respaldo, o comportamiento degradado definido. El fallo de un agente no colapsa el sistema completo.
- [] **Timeouts definidos.** Cada espera del sistema tiene un tiempo máximo. No hay esperas indefinidas.

- [] **Idempotencia en entregas duplicadas.** Si un mensaje es entregado dos veces, el procesamiento produce el mismo resultado en ambas entregas. El sistema verifica identificadores de mensaje antes de procesar.
 - [] **Observabilidad implementada.** El sistema registra, como mínimo: cada mensaje enviado y recibido entre agentes (con marca temporal), el estado de cada subtarea (iniciada, completada, con error) y el costo de tokens por agente y por tarea.
 - [] **Límites de reintentos definidos.** Hay un número máximo de reintentos para cada agente. Cuando se alcanza ese límite, el sistema reporta el fallo en lugar de continuar reintentando indefinidamente.
-

Bloque 6: Costo y viabilidad

- [] **Estimación de costo por tarea.** Existe una estimación del número de llamadas al modelo de lenguaje que realiza el sistema por tarea, y del costo total de tokens esperado.
 - [] **Estimación de latencia.** Existe una estimación de la latencia total esperada por tarea, considerando el paralelismo y los tiempos de cada agente.
 - [] **Costo justificado por valor.** El costo adicional de la arquitectura multiagente (versus un agente único) está justificado por el valor que añade: menor latencia, mayor calidad, mayor cobertura. Ese análisis se hizo explícitamente.
 - [] **Plan de monitoreo de costo en producción.** Hay un mecanismo para detectar cuando el costo real en producción se desvía significativamente de la estimación. Las desviaciones grandes activan una revisión del diseño.
-

La sección 13 resume los conceptos centrales del capítulo en una visión integrada. Un buen resumen no es una lista de lo que se cubrió: es la articulación del marco mental que el lector debe llevarse.

Capítulo 09 — Arquitecturas Multiagente

Sección 13 — Resumen del capítulo

Este capítulo construyó el marco completo para diseñar sistemas multiagente desde la decisión de arquitectura hasta la operación en producción. El hilo conductor es una idea simple pero no trivial: la arquitectura multiagente no es una versión mejorada del agente único. Es una respuesta diferente a una clase diferente de problemas.

El marco mental que este capítulo establece

El multiagente se justifica por el problema, no por la tecnología. Hay cuatro condiciones que justifican una arquitectura multiagente: necesidad de paralelismo genuino, necesidad de especialización radicalmente distinta por dominio, necesidad de verificación independiente por criticidad del output, y necesidad de distribuir información que excede la ventana de contexto de un agente único. Cuando ninguna de estas condiciones está presente, el agente único es la arquitectura correcta, no el multiagente.

La especialización de cada agente determina la calidad del sistema. Un sistema multiagente con agentes mal definidos produce resultados de menor calidad que un agente único bien diseñado. La especialización no es un detalle de implementación: es el mecanismo principal de valor de la arquitectura. Un agente especializado tiene un rol limitado, herramientas mínimas necesarias y una instrucción de sistema orientada exclusivamente a su función.

La topología es la decisión arquitectónica más importante después de la especialización. La topología jerárquica centraliza la coordinación y facilita el control; la pipeline impone secuencialidad natural; la topología entre pares permite adaptación descentralizada; la basada en mercado distribuye carga dinámicamente. Cada topología tiene casos de uso óptimos y compromisos específicos. Los sistemas reales frecuentemente combinan varias.

La comunicación entre agentes debe ser estructurada y resiliente. Los mensajes entre agentes tienen un esquema definido, con tipo, identificador, payload y metadatos. La

comunicación puede ser síncrona o asíncrona según si el emisor necesita el resultado del receptor antes de continuar. Todo mecanismo de comunicación debe estar diseñado para manejar mensajes malformados, pérdidas y entregas duplicadas.

El planificador y el supervisor son los agentes de control del sistema. El planificador transforma una tarea compleja en un plan de subtareas con dependencias explícitas. El supervisor verifica que los outputs de los agentes ejecutores cumplan criterios de calidad antes de que sean aceptados. Sin planificación, el sistema no puede manejar tareas no completamente anticipadas. Sin supervisión, los errores se propagan hasta el output final.

La memoria compartida es un problema de ingeniería de sistemas, no de IA. Cuando múltiples agentes acceden y modifican el mismo estado, se producen condiciones de carrera que generan inconsistencias. Las estrategias para manejar este problema —escrituras atómicas, control de concurrencia optimista, paso de mensajes— tienen cada una sus compromisos de consistencia y rendimiento. El diseñador del sistema debe elegir la estrategia apropiada y aplicarla consistentemente.

La tolerancia a fallos no es opcional en producción. Los fallos en sistemas multiagente son más frecuentes que en sistemas de agente único porque hay más componentes que pueden fallar. Los reintentos con backoff, los agentes de respaldo, los circuit breakers y los puntos de verificación son estrategias de resiliencia que deben diseñarse antes de la primera línea de código de producción. La observabilidad completa del sistema —trazas de mensajes, estados de subtareas, métricas de costo— es la condición necesaria para operar el sistema y detectar problemas antes de que sean reportados por usuarios.

Los patrones y anti-patrones nombran lo que ya se sabe que funciona y lo que ya se sabe que falla. El patrón de Reflexión mejora la calidad mediante verificación independiente. El patrón de Ejecución Paralela con Síntesis reduce la latencia en tareas descomponibles. El patrón de Escalada Supervisada reduce el costo promedio concentrando el uso del agente más capaz en los casos más complejos. El anti-patrón del Agente Dios produce sistemas difíciles de mantener. La Red de Charlas produce sistemas ineficientes. La Confianza No Verificada produce sistemas frágiles ante errores en agentes upstream. El Multiagente Innecesario produce sistemas costosos donde no había necesidad de complejidad.

Lo que distingue un sistema multiagente bien diseñado

Un sistema multiagente bien diseñado tiene las siguientes propiedades observables:

- Los roles de los agentes son precisos: cualquier persona que lee la instrucción de sistema de un agente entiende exactamente qué hace y qué no hace.
- La topología refleja la naturaleza del problema: las partes paralelas del trabajo se ejecutan en paralelo, las secuenciales en secuencia, y hay un mecanismo de control que garantiza que las dependencias se respetan.
- Los fallos son detectables y recuperables: cuando algo falla, el sistema lo detecta, reporta y recupera sin perder el trabajo completado hasta ese punto.
- El costo está justificado por el valor: la diferencia de calidad o velocidad que produce el sistema multiagente sobre un agente único justifica el costo adicional de tokens, latencia y complejidad operacional.

Lo que este capítulo no cubre

Un sistema multiagente bien diseñado es un componente de un sistema más amplio. Este capítulo desarrolló la arquitectura del sistema multiagente en sí. El Módulo 4 del libro amplía la perspectiva hacia los sistemas completos donde los agentes operan: la infraestructura de orquestación a escala, los mecanismos de evaluación continua, la integración con sistemas de software empresariales y los modelos de gobierno y auditoría que los sistemas de IA en producción requieren.

La transición de los fundamentos a la arquitectura de sistema completo es el paso siguiente. El capítulo 10 de este módulo establece el puente entre ambos: el problema del razonamiento y la planificación, que es la capacidad que permite a un sistema multiagente —o a un agente único— resolver problemas que no fueron completamente anticipados en su diseño.

La sección 14 pone a prueba la comprensión del capítulo con preguntas que exigen razonamiento, no memorización. Las respuestas no están explícitamente en el texto: emergen de la aplicación de los marcos desarrollados a situaciones nuevas.

Capítulo 09 — Arquitecturas Multiagente

Sección 14 — Autoevaluación

Las preguntas de esta sección no tienen respuestas de una palabra. Son preguntas de razonamiento: requieren aplicar los marcos del capítulo a situaciones que no aparecieron exactamente en el texto. Si puedes responderlas con precisión, has construido el modelo mental correcto. Si alguna revela una laguna, esa laguna señala exactamente la sección del capítulo que vale la pena releer.

Pregunta 1

Un equipo propone un sistema multiagente con doce agentes para automatizar la generación de informes de ventas semanales. Los informes tienen siempre la misma estructura, se basan en datos que el sistema puede consultar directamente desde la base de datos de ventas, y no requieren análisis subjetivo ni síntesis de múltiples fuentes heterogéneas.

¿Qué le dirías al equipo? Aplica el árbol de decisión de la sección 02 al caso y justifica tu recomendación.

Pregunta 2

Estás diseñando un sistema multiagente para analizar contratos legales. El sistema tiene un agente extractor (identifica las cláusulas relevantes), un agente analista (evalúa las cláusulas contra los criterios de la empresa) y un agente redactor (produce el resumen ejecutivo).

El agente analista necesita, para algunos contratos, consultar precedentes de contratos anteriores de la empresa. Esos precedentes están en la memoria compartida del sistema.

¿Qué consideraciones de diseño aplicas para que el agente analista acceda a los precedentes de forma correcta y segura? Incluye consideraciones de consistencia, aislamiento y privacidad.

Pregunta 3

Tu sistema multiagente lleva tres semanas en producción. El costo de tokens es un 40% mayor de lo esperado en la estimación inicial. El output de calidad es el esperado. ¿Cuáles son las causas más probables de esa desviación y cómo la investigarías? ¿Qué cambios de diseño evaluarías?

Pregunta 4

Tienes un sistema con la siguiente topología: un agente orquestador que distribuye tareas entre cuatro agentes especializados, que devuelven sus resultados al orquestador, que luego los envía a un agente redactor. Es una topología jerárquica clásica.

En producción, el sistema falla con frecuencia en el paso de síntesis del orquestador: los cuatro resultados de los agentes especializados son correctos individualmente, pero el orquestador tiene dificultades para integrarlos de forma coherente porque los formatos de output de los cuatro agentes son distintos entre sí.

Diagnostica el problema y propón dos alternativas de solución con sus compromisos.

Pregunta 5

Un colega propone añadir al sistema del caso de estudio de la sección 10 (evaluación de propuestas de proveedores) un agente de negociación que, después de producir el borrador de evaluación, genere automáticamente una contraoferta al proveedor si las condiciones comerciales no cumplen los parámetros de la empresa.

Evalúa esta propuesta. ¿Qué dimensiones del diseño cambian? ¿Qué consideraciones nuevas aparecen que no estaban presentes en el diseño original?

Pregunta 6

Describe la diferencia entre el anti-patrón del Agente Dios y un agente orquestador legítimo. ¿Qué características tiene el orquestador que lo distinguen del Agente Dios? ¿Cómo sabrías

cuándo un orquestador está convirtiéndose en un Agente Dios?

Pregunta 7

Tu sistema multiagente tiene un agente supervisor que evalúa el output del agente redactor antes de entregarlo al usuario. La tasa de rechazo del supervisor es del 35%: rechaza un tercio de los borradores que recibe y los devuelve al redactor para corrección.

Esa tasa te parece alta. ¿Significa que el sistema está funcionando mal? ¿O podría ser una señal de que el sistema está funcionando correctamente? ¿Qué información adicional necesitarías para distinguir entre ambos casos?

Pregunta 8

Diseña brevemente la estrategia de resiliencia para el siguiente escenario: un sistema de pipeline en el que cinco agentes actúan en secuencia ($A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$) para transformar un documento desde su forma original hasta su forma final. El proceso completo toma en promedio ocho minutos.

¿Qué ocurre si el agente C falla a los cuatro minutos de procesamiento? ¿Cómo diseñas el sistema para que no sea necesario reiniciar desde A?

Criterios de autodiagnóstico

Si respondiste las preguntas 1, 2 y 6 con claridad y precisión: tienes sólida comprensión de los conceptos fundamentales del capítulo (cuándo usar multiagente, diseño de agentes, patrones y anti-patrones).

Si también respondiste las preguntas 3, 4 y 8: tienes comprensión operacional del capítulo (costo, debugging, resiliencia).

Si también respondiste las preguntas 5 y 7: tienes comprensión de las dimensiones de diseño más sutiles del capítulo (cambios de alcance, interpretación de métricas de sistema).

Si alguna pregunta quedó sin respuesta satisfactoria, ese es el punto de entrada para releer el capítulo con un objetivo específico, no una relectura completa.

La sección 15 establece el puente entre los sistemas multiagente de este capítulo y el problema del razonamiento y la planificación que define el capítulo 10.

Capítulo 09 — Arquitecturas Multiagente

Sección 15 — Transición al Capítulo 10

A lo largo de este capítulo diseñamos sistemas en los que múltiples agentes colaboran para resolver problemas que un agente único no puede resolver bien. Definimos roles, elegimos topologías, diseñamos protocolos de comunicación, gestionamos estado compartido y construimos resiliencia ante fallos. Todo eso es la arquitectura del sistema.

Pero hay un presupuesto implícito en todo lo que hemos diseñado: que los agentes —tanto el individual como los que componen un sistema multiagente— pueden tomar decisiones correctas sobre qué hacer a continuación. Que cuando el planificador descompone una tarea, lo hace de forma que el plan resultante realmente lleva a la solución del problema. Que cuando el orquestador asigna subtareas, las asigna a los agentes correctos en el orden correcto. Que cuando un agente encuentra un obstáculo en su ejecución, puede razonar sobre cómo superarlo.

Ese presupuesto es el razonamiento y la planificación. Y hasta ahora lo hemos asumido sin examinarlo.

Lo que queda sin explicar

¿Cómo decide un agente orquestador que la subtarea A debe ejecutarse antes que la B? ¿Cómo determina el planificador que el problema original puede descomponerse en estas tres subtareas y no en cinco distintas? ¿Qué ocurre cuando el plan inicial resulta incorrecto a la mitad de la ejecución? ¿Cómo reconoce el sistema que el camino actual no lleva al objetivo y necesita explorar una alternativa?

Estas preguntas no tienen respuesta en la arquitectura del sistema. Tienen respuesta en la capacidad de razonamiento de los agentes que lo componen.

El problema del razonamiento en sistemas de IA

Un agente que recibe una tarea no trivial necesita hacer más que ejecutar instrucciones predefinidas. Necesita comprender el objetivo, identificar los pasos que llevan a ese objetivo,

anticipar los obstáculos que pueden aparecer en cada paso, adaptar el plan cuando la realidad no coincide con lo anticipado y evaluar si el resultado final cumple el objetivo original.

Esta capacidad —razonar sobre cómo llegar a un objetivo— es lo que distingue un agente de IA de un workflow automatizado. Un workflow sigue reglas fijas. Un agente razona sobre la situación y decide. Esa diferencia tiene consecuencias directas en qué tipos de problemas puede resolver cada sistema.

Los sistemas multiagente del capítulo 09 son poderosos precisamente porque distribuyen esta capacidad de razonamiento entre múltiples agentes especializados. Pero el razonamiento de cada agente individualmente —y del planificador en particular— es el componente que determina si el sistema puede manejar la variabilidad del mundo real.

Lo que el Capítulo 10 desarrolla

El Capítulo 10 examina el razonamiento y la planificación como disciplinas de diseño:

Cómo los modelos de lenguaje razonan. Los mecanismos por los cuales un modelo de lenguaje puede articular pasos de razonamiento explícitos —cadenas de pensamiento, razonamiento paso a paso, verificación de consistencia— y cómo esos mecanismos mejoran la calidad de las decisiones en comparación con respuestas directas sin razonamiento articulado.

Cómo diseñar agentes planificadores efectivos. Las técnicas de prompting y arquitectura que permiten a un agente descomponer tareas complejas en planes ejecutables: qué información necesita el planificador, cómo se estructura su instrucción de sistema, cómo se valida que el plan es coherente y completo antes de ejecutarlo.

Cómo manejar la incertidumbre y la revisión del plan. Los sistemas que operan en el mundo real encuentran situaciones que el plan no anticipó. El razonamiento adaptativo —la capacidad de un agente de reconocer que el plan actual no funciona y generar un plan alternativo— es la propiedad que hace que un sistema sea genuinamente robusto y no simplemente funcional en el caso nominal.

Los límites del razonamiento actual. No todos los problemas son razonables para los sistemas de IA actuales. Hay tipos de problemas donde el razonamiento de los modelos de lenguaje falla de forma predecible. Conocer esos límites es tan importante como conocer las capacidades: permite diseñar sistemas que delegan a la IA lo que la IA puede hacer bien y mantienen control humano sobre lo que no puede.

La continuidad entre ambos capítulos

Los sistemas multiagente de este capítulo y los mecanismos de razonamiento del capítulo 10 no son temas separados. Son capas del mismo sistema: la arquitectura multiagente define cómo está organizado el sistema; el razonamiento define cómo cada agente dentro de ese sistema toma decisiones efectivas.

Un sistema multiagente sin razonamiento sólido en sus agentes es un sistema bien organizado que produce resultados mediocres. Un agente con razonamiento sólido pero sin la arquitectura adecuada para el problema no puede escalar ni especializarse lo suficiente para resolver problemas complejos. La combinación de ambos —arquitectura multiagente diseñada con criterio y agentes con razonamiento efectivo— es la base de los sistemas de IA de producción más capaces que existen hoy.

El Capítulo 10 construye la segunda mitad de esa combinación.

El Capítulo 10 comienza examinando cómo los modelos de lenguaje generan razonamiento articulado, por qué ese razonamiento articulado produce mejores resultados que el razonamiento implícito, y qué técnicas de diseño permiten al AI Engineer aprovechar esa capacidad de forma sistemática y predecible.